

Logistic Regression

Trần Hải Nam (B24DCCN418)

figures/logo.png

Bộ môn XLTH & TT
Khoa Kỹ thuật Điện tử 1
Học viện Công nghệ Bưu chính Viễn thông

Ngày 13 tháng 05 năm 2026

Mục lục

1 Logistic Regression

figures/logo.png

Tổng quan và Bài toán áp dụng

- **Bản chất:** Là mô hình **phân loại tuyến tính** (Linear Classification). Không phải bài toán hồi quy giá trị liên tục. Kết quả đầu ra là xác suất.
- **Bài toán cốt lõi:** Phân loại nhị phân (Binary Classification). Phân chia dữ liệu thành hai lớp: 0 hoặc 1.
- **Mở rộng đa lớp (OvR):** Chia bài toán N lớp thành N bài toán nhị phân.
- **Mở rộng đa lớp (Softmax):** Tính trực tiếp xác suất của tất cả các lớp với:

$$\sum_{i=1}^N P(y = i|x) = 1$$

Cơ chế hoạt động

Bước 1: Chấm điểm thô

$$s = w^T x + b$$

Nhập dữ liệu đầu vào với trọng số. Điểm thô nằm trong khoảng $(-\infty, +\infty)$.

Bước 2: Hàm Sigmoid

$$\sigma(s) = \frac{1}{1 + e^{-s}}$$

Chuyển điểm thô thành xác suất trong khoảng $(0, 1)$.

Bước 3: Phân loại So sánh với ngưỡng (Threshold), mặc định là 0.5. Có thể thay đổi tùy yêu cầu bài toán.

Hàm mất mát và Tối ưu tham số

Hàm mất mát (Log Loss / Binary Cross-Entropy)

$$J(w) = - \sum_{i=1}^N (y_i \log z_i + (1 - y_i) \log(1 - z_i))$$

Đo mức sai lệch giữa dự đoán và thực tế. Phạt rất nặng khi mô hình dự đoán sai nhưng quá tự tin.

Tối ưu tham số (Gradient Descent)

- Tính Gradient của hàm mất mát.
- Cập nhật trọng số: $w \leftarrow w - \alpha \nabla J(w)$
- Lặp cho đến khi hội tụ.

Tổng quan Softmax Regression

1. Khái niệm & Ứng dụng

- Là phiên bản mở rộng của Logistic Regression.
- **Loại bài toán:** Phân loại đa lớp (Multi-class Classification) với $C \geq 3$ lớp.
- **Điều kiện:** Các lớp mang tính loại trừ lẫn nhau (Mutually Exclusive – một dữ liệu chỉ thuộc đúng 1 lớp).

Cơ chế hoạt động của Softmax Regression

Bước 1: Tính điểm số thô (Logits)

- Tính điểm số cho mỗi lớp bằng phép nhân tuyến tính:

$$z = W^T x$$

Bước 2: Chuyển đổi sang xác suất

- Đưa các điểm số qua hàm Softmax để tạo phân phối xác suất (tổng bằng 1):

$$a_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Bước 3: Dự đoán

- Dự đoán nhãn có xác suất a_i cao nhất.

Hàm mất mát (Loss Function)

1. Nền tảng: Cross-Entropy

- Đo lường khoảng cách giữa hai phân phối xác suất rời rạc p và q :

$$H(p, q) = - \sum_{i=1}^C p_i \log q_i \quad (1)$$

figures/logo.png

2. Áp dụng vào Softmax Regression

- Với ma trận trọng số W cần tối ưu, hàm mất mát cho toàn bộ N điểm dữ liệu (x_i, y_i) là:

$$J(W; X, Y) = - \sum_{i=1}^N \sum_{j=1}^C y_{ji} \log(a_{ji})$$

Hàm mất mát Softmax Regression

Hàm mất mát cho toàn bộ tập dữ liệu

$$J(W; X, Y) = - \sum_{i=1}^N \sum_{j=1}^C y_{ji} \log \left(\frac{\exp(w_j^T x_i)}{\sum_{k=1}^C \exp(w_k^T x_i)} \right)$$

figures/logo.png

- W : Ma trận trọng số cần tối ưu.
- N : Số lượng điểm dữ liệu.
- C : Số lượng lớp.
- y_{ji} : Nhãn thực tế của mẫu i thuộc lớp j .
- a_{ji} : Xác suất dự đoán mẫu i thuộc lớp j .

Tối ưu hóa với SGD – Phần 1: Rút gọn

1. Hàm mất mát cho 1 điểm dữ liệu

- Thay vì tính toàn bộ, SGD xét từng điểm (x_i, y_i) :

$$J_i(W) = - \sum_{j=1}^C y_{ji} \log \left(\frac{\exp(w_j^T x_i)}{\sum_{k=1}^C \exp(w_k^T x_i)} \right)$$

2. Rút gọn phương trình

- Áp dụng tính chất:

$$\sum_{j=1}^C y_{ji} = 1$$

Tối ưu hóa với SGD – Rút gọn hàm mất mát

$$J_i(W) = - \sum_{j=1}^C \left(y_{ji} w_j^T x_i - y_{ji} \log \left(\sum_{k=1}^C \exp(w_k^T x_i) \right) \right)$$
$$= - \sum_{j=1}^C y_{ji} w_j^T x_i + \log \left(\sum_{k=1}^C \exp(w_k^T x_i) \right)$$

- Thành phần thứ nhất biểu diễn ảnh hưởng của trọng số đối với nhãn thực tế.
- Thành phần thứ hai là hàm log của tổng các giá trị exp từ các lớp.

Tối ưu hóa với SGD – Phần 2: Đạo hàm

1. Đạo hàm (Gradient)

- Đạo hàm theo từng cột trọng số w_j (với e_{ji} là sai số dự đoán):

$$\frac{\partial J_i(W)}{\partial w_j} = x_i(a_{ji} - y_{ji}) = e_{ji}x_i \quad (5)$$

- Đạo hàm theo ma trận W cho 1 điểm dữ liệu:

$$\frac{\partial J_i(W)}{\partial W} = x_i e_i^T$$

- Suy rộng cho toàn bộ tập dữ liệu:

$$\frac{\partial J(W)}{\partial W} = XE^T$$

Tối ưu hóa với SGD – Phần 2: Cập nhật

2. Công thức cập nhật (SGD)

- Sau khi tính được gradient, ta cập nhật ma trận trọng số theo hướng ngược với gradient.

$$W = W - \eta \frac{\partial J_i(W)}{\partial W}$$

figures/logo.png

- Với η là learning rate, sử dụng:

$$W = W + \eta x_i (y_i - a_i)^T$$

- Trọng số được điều chỉnh để giảm sai số dự đoán sau mỗi lần cập nhật.

Sự hạn chế của Accuracy & Ngưỡng mặc định

Điểm mù của Accuracy:

- Vô dụng với dữ liệu mất cân bằng.
- Ví dụ: 99% khỏe, 1% bệnh. Mô hình đoán bừa “Khỏe” cho tất cả vẫn đạt Accuracy 99%, nhưng bỏ lọt 100% người bệnh.

Hạn chế của ngưỡng mặc định (0.5):

- Mô hình trả về xác suất liên tục:

$$P \in [0, 1]$$

- Mốc 0.5 chỉ là cân bằng toán học, không phản ánh rủi ro thực tế.
- Thực tế: Y tế cần “thà bắt nhầm còn hơn bỏ sót”.

⇒ Bắt buộc phải tự tinh chỉnh ngưỡng.

Ma trận nhầm lẫn (Confusion Matrix)

Các thành phần của Confusion Matrix

- **True Positive (TP):** Lớp thực tế là Positive → Mô hình dự đoán **ĐÚNG** là Positive.
- **True Negative (TN):** Lớp thực tế là Negative → Mô hình dự đoán **ĐÚNG** là Negative.
- **False Positive (FP):** Lớp thực tế là Negative → Mô hình dự đoán **SAI** thành Positive. (*Báo động giả / Sai số loại I*).
- **False Negative (FN):** Lớp thực tế là Positive → Mô hình dự đoán **SAI** thành Negative. (*Bỏ lọt mục tiêu / Sai số loại II*).

		Predicted values	
		Positive (cat)	Negative (not cat)
Actual values	Positive (cat)	True positive  This is a cat.	False negative  This is not a cat. Type 2 error
	Negative (not cat)	False positive  This is a cat. Type 1 error	True negative  This is not a cat.

Precision & Recall

Precision (Độ chính xác)

- **Khẩu quyết:** “Trong số những điểm được phân loại là Positive, có bao nhiêu điểm là True Positive (đúng)?”

$$\text{Precision} = \frac{TP}{TP + FP}$$

Ý nghĩa: Precision cao đồng nghĩa với việc độ chính xác của các điểm tìm được là cao.

Recall (Độ phủ / True Positive Rate)

- **Khẩu quyết:** “Trong số những điểm thực sự là Positive, có bao nhiêu điểm là True Positive (đúng)?”

$$\text{Recall} = \frac{TP}{TP + FN}$$

Ý nghĩa: Recall cao đồng nghĩa với việc tỉ lệ bỏ sót các điểm thực sự Positive là thấp.

F1-Score (Chỉ số hài hòa)

Tại sao cần F1-Score?

Để tránh việc mô hình đánh lừa bằng cách hy sinh hoàn toàn một chỉ số để đẩy chỉ số còn lại lên mức 100%.

Công thức

Là trung bình điều hòa (Harmonic Mean) của Precision và Recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Đặc điểm

- Mô hình bị “trừng phạt” rất nặng nếu một trong hai chỉ số có giá trị quá thấp.
- Chỉ đạt điểm cao khi **cả Precision và Recall** đều ở mức cao.

Kết luận

F1-Score là thước đo tiêu chuẩn để đánh giá hiệu năng tổng thể của mô hình trên dữ liệu mất cân bằng.

Tối ưu ngưỡng với Đồ thị ROC & AUC

Bản chất

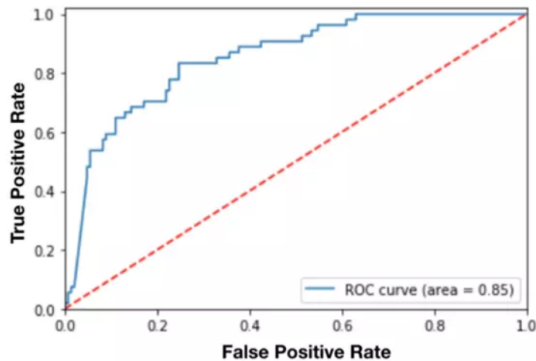
- Không thể “thử mò” ngưỡng.
- Phải đánh giá mô hình trên nhiều mức ngưỡng.

Đồ thị ROC

- Quan hệ giữa TPR và FPR.
- Khi thay đổi ngưỡng phân loại.

Chỉ số AUC

- AUC là diện tích dưới đường cong ROC.
- Đánh giá khả năng phân biệt giữa các lớp.
- AUC càng lớn, mô hình càng tốt.



Kỹ thuật chốt ngưỡng

- Không có ngưỡng phân loại nào là **“hoàn hảo”** tuyệt đối về toán học.
- Việc chốt **Threshold** bị chi phối hoàn toàn bởi mức độ rủi ro và chi phí của bài toán.

Kịch bản 1: Ưu tiên tối đa Recall

- **Mục tiêu:** “Thà bắt nhầm còn hơn bỏ sót”.
- **Hành động:** Hạ ngưỡng (VD: ép $Recall > 98\%$), chấp nhận nhiều báo động giả.
- **Ứng dụng:** Y tế (chẩn đoán bệnh), An ninh.

Kịch bản 2: Ưu tiên tối đa Precision

- **Mục tiêu:** “Đã cảnh báo là phải chắc chắn đúng, cấm xử oan”.
- **Hành động:** Tăng ngưỡng (VD: ép $Precision > 95\%$), chấp nhận bỏ lọt mục tiêu.
- **Ứng dụng:** Khóa tài khoản tự động, Lọc Spam.

Kết luận: Threshold là một **“núm xoay”** để kỹ sư chủ động kiểm soát sự đánh đổi (**Trade-off**) theo đúng yêu cầu nghiệp vụ.

Tại sao PR Curve "đánh bại" ROC ở Đa lớp?

Bẫy OvR (One-vs-Rest)

- Gom $N - 1$ lớp còn lại thành lớp **Negative**.
- Tạo ra lượng **True Negative (TN)** khổng lồ.

ROC "ảo tưởng"

- Công thức tính **FPR** chứa TN ở mẫu số.
- Điểm số dễ bị nhiễu, tạo ra kết quả cao giả tạo.

PR Curve

- Precision và Recall không chứa TN trong công thức.
- Không bị ảnh hưởng bởi dữ liệu mất cân bằng.
- Phản ánh đúng hiệu năng thực sự của mô hình.

Kết luận

Micro-average và Macro-average

1. Micro-average (Gom chung một rổ)

- Bản chất:** “Bình đẳng Dữ liệu” (1 mẫu = 1 phiếu bầu). Xóa bỏ ranh giới lớp, chỉ quan tâm tổng số lần đoán trúng.

$$Micro_P = \frac{\sum TP}{\sum (TP + FP)}$$

2. Macro-average (Trung bình từng môn)

- Bản chất:** “Bình đẳng Tập thể” (1 lớp = 1 phiếu bầu). Tính điểm độc lập cho từng lớp rồi lấy trung bình.

$$Macro_P = \frac{P_1 + P_2 + \dots + P_C}{C}$$

(C là số lượng lớp)

Ví dụ trực quan Micro vs. Macro

Bài toán:

Nhận diện 100 quả Táo, 10 quả Cam, 10 quả Nho.

Điểm riêng:

Táo (90%), Cam (50%), Nho (10%).

Sự khác biệt

Tính kiểu Micro = 80%

- Lớp Táo đông dân đã “gánh team”.
- Con số 80% che lấp sự thật là hệ thống dự đoán Nho sai bét nhè.

Tính kiểu Macro = 50%

- Bị kéo tụt bởi điểm 10% của lớp Nho.
- Bóc trần hiệu năng thực sự, bắt buộc kỹ sư phải tinh chỉnh lại mô hình.

figures/logo.png